

HMIS

Hankes Music Intelligence System

Document d'Architecture Technique

Version	1.0 — Avril 2026
Statut	Confidentiel — Usage reserve a HANKES
Auteur	Equipe Technique HMIS
Perimetre	Backend, Mobile, Frontend, Infra, Securite
Technologies	NestJS, React Native, React, PostgreSQL, Docker, AWS/GCP

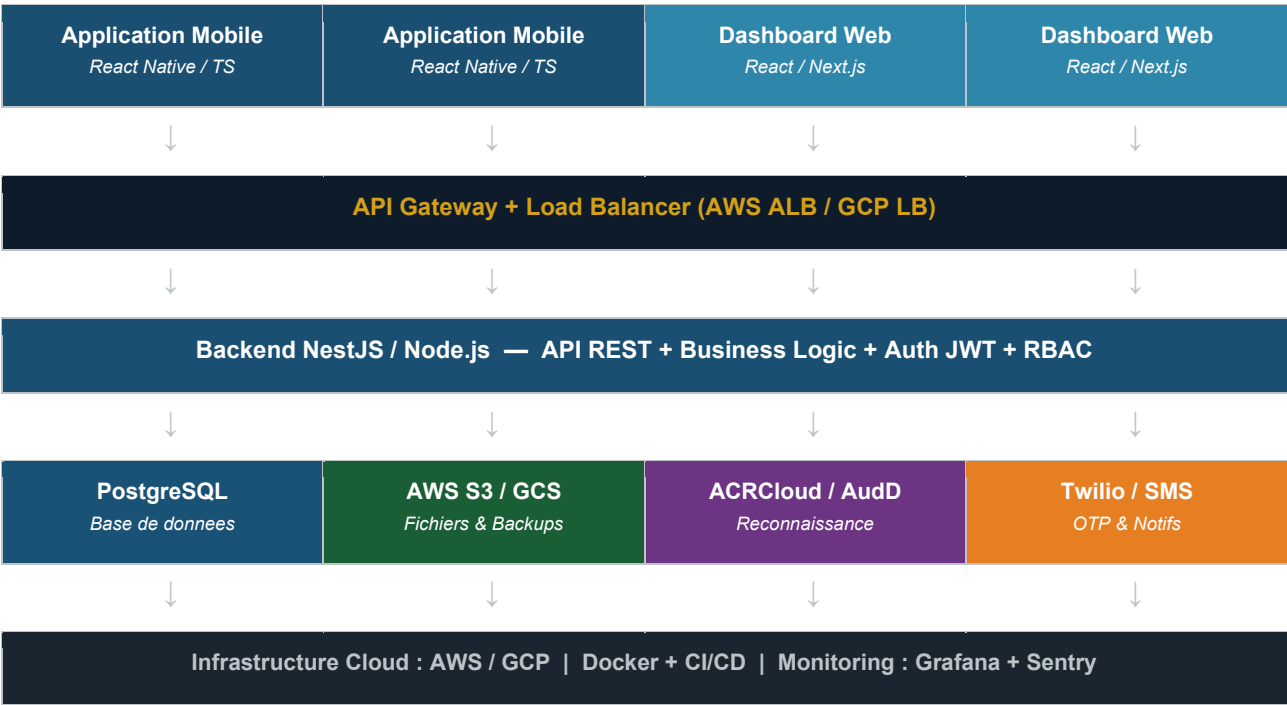
SOMMAIRE

1. Vue d'ensemble de l'architecture
2. Architecture applicative — Couches et composants
3. Backend NestJS — Structure et modules
4. Application Mobile React Native — Architecture offline-first
5. Frontend Web — Dashboard et reporting
6. Base de donnees PostgreSQL — Modele de donnees
7. Infrastructure Cloud — AWS / Google Cloud
8. Securite et authentication
9. Integration API de reconnaissance musicale
10. Pipeline CI/CD et DevOps
11. Monitoring, logs et alertes
12. Flux de donnees complets
13. Decisions d'architecture — ADR
14. Glossaire technique

1. Vue d'ensemble de l'Architecture

Le systeme HMIS repose sur une architecture trois-tiers moderne, cloud-native et orientee microservices. Chaque couche est independante, testable et deployable de facon autonome, garantissant une evolvabilite et une maintenabilite optimales sur le long terme.

1.1 Diagramme general



Ce schema illustre les cinq couches principales du systeme et les interactions entre elles. Les fleches representent des appels HTTPS securises. Chaque couche est isolee et communique uniquement via des interfaces contractualisees (API REST, ORM).

1.2 Principes architecturaux fondamentaux

Principe	Description et application dans HMIS
Separation des preoccupations	Chaque composant a une responsabilite unique : l'app mobile capture, le backend orchestre, la BDD stocke, le dashboard visualise.
Offline-first	L'application mobile fonctionne sans connexion Internet. Les captures sont stockees localement et synchronisees a la reconnexion.
Securite by-design	Chiffrement TLS 1.3, JWT, RBAC, validation des entrees, rate limiting : la securite est integree des la conception, pas en couche supplementaire.
Scalabilite horizontale	Le backend NestJS est stateless et peut etre replique. Le load balancer AWS ALB distribue le trafic. La BDD peut etre scalee via RDS read replicas.
Observabilite	Chaque composant emet des metriques, logs et traces. Sentry capture les erreurs, Grafana visualise les metriques, CloudWatch surveille l'infrastructure.

Principe	Description et application dans HMIS
Infrastructure as Code	L'infrastructure cloud est declaree et versionnable via Terraform, garantissant des environnements reproductibles.
12-Factor App	L'application suit les 12 principes de l'app cloud-native : config par variables d'environnement, logs vers stdout, processus stateless, etc.

1.3 Environnements

Environnement	URL / Description	Usage
Developpement	http://localhost:3000	Developpement local, Docker Compose
Staging	https://staging-api.hmis-project.ci	Recette, tests integration, validation client
Production	https://api.hmis-project.ci	Environnement live — trafic reel

2. Architecture Applicative — Couches et Composants

2.1 Couche Client (Tier 1)

La couche client englobe tous les points d'entree humains du systeme : l'application mobile utilisee par les etablissements, et le dashboard web utilise par les administrateurs HANKES et les partenaires institutionnels.

Client	Technologie	Public cible	Fonctionnalites principales
App Mobile	React Native 0.73+ (TypeScript)	Gerants d'etablissements	Capture audio passive, onboarding, profil, historique, mode offline
Dashboard Web	Next.js 14 + React 18	Admin HANKES, Partenaires (BURIDA)	KPIs, carte geo, top musiques, reporting, gestion etablissements

2.2 Couche API / Metier (Tier 2)

Le backend constitue le coeur du systeme. Il est entierement developpe en NestJS (framework Node.js) et expose une API REST completement documentee via Swagger. Il est responsable de la logique metier, de la securite et de l'orchestration des services externes.

Responsabilites du Backend NestJS

- Authentification et gestion des sessions (JWT + Refresh Token + OTP)
- Gestion des roles et permissions (RBAC : admin, etablissement, partenaire)
- Orchestration de la reconnaissance musicale (appels ACRCLOUD/AuDD)
- Stockage et interrogation des logs de diffusion (via TypeORM + PostgreSQL)
- Generation des rapports (PDF via PDFKit, Excel via ExcelJS)
- Exposition de l'API REST documentee (Swagger / OpenAPI 3.0)
- Rate limiting, validation des entrees, gestion des erreurs centralisee
- Emission des metriques et logs vers les outils de monitoring

2.3 Couche Donnees (Tier 3)

La couche donnees est composee de plusieurs services de persistance, chacun specialise dans un type de donnee :

Service	Technologie	Donnees stockees	Justification
Base relationnelle	PostgreSQL 15 (AWS RDS)	Utilisateurs, etablissements, diffusions, logs	Donnees structurees, requetes analytiques complexes, ACID
Stockage objets	AWS S3 / Google Cloud Storage	Extraits audio temporaires, rapports PDF/Excel, backups	Stockage blob economique et durable, CDN possible
Cache (futur)	Redis (optionnel Phase 2)	Cache requetes API, sessions, rate limiting	Reduction latence dashboard, decharge BDD

3. Backend NestJS — Structure et Modules

3.1 Structure du projet

Le backend suit la structure modulaire imposée par NestJS. Chaque module encapsule ses propres controllers, services, DTOs et entités. Cette organisation garantit une séparation claire des responsabilités et facilite les tests unitaires.

```
src/
  auth/      # JWT, OTP, guards, strategies
  users/     # CRUD utilisateurs, profils
  etablisements/ # CRUD, validation, statut
  audio/     # Upload, sync offline, config
  reconnaissance/ # Integration ACRCLOUD/AudD
  diffusions/ # Logs, historique, statistiques
  dashboard/ # KPIs, top musiques, carte geo
  rapports/  # Generation PDF/Excel, jobs asynchrones
  common/   # Guards, pipes, interceptors, decorators
  config/   # Variables d'environnement, validation
  database/  # TypeORM, migrations, seeders
```

3.2 Modules et responsabilités

Module	Controllers	Services principaux	Dependances externes
auth	POST /login, /register, /otp, /refresh	AuthService, OtpService, TokenService	JWT, Twilio, Passport.js
users	GET/PATCH /utilisateurs	UserService	TypeORM, bcrypt
etablisements	CRUD /etablisements	EtablisementsService, ValidationService	TypeORM, Mailer
audio	POST /audio/capturer, /sync; GET /config	AudioService, SyncQueueService	Multer, AWS S3, Bull Queue
reconnaissance	Interne (appelé par AudioService)	ReconnaissanceService, AcrCloudAdapter	ACRCLOUD API, AudD API
diffusions	GET /diffusions, /etablisements/:id/diffusions	DiffusionsService	TypeORM, PostgreSQL
dashboard	GET /dashboard/kpis, /top-*, /carte, /evolution	DashboardService, StatsService	TypeORM, Redis (cache)
rapports	POST/GET /rapports	RapportsService, PdfGeneratorService, ExcelService	PDFKit, ExcelJS, Bull Queue, S3

3.3 Middlewares et intercepteurs

Composant	Type	Role
JwtAuthGuard	Guard global	Vérifie la validité du JWT sur tous les endpoints protégés

Composant	Type	Role
RolesGuard	Guard	Verifie que le role de l'utilisateur autorise l'action (RBAC)
ValidationPipe global	Pipe	Valide et transforme automatiquement tous les DTOs entrants (class-validator)
LoggingInterceptor	Interceptor	Logue chaque requete : methode, path, duree, statut HTTP
TransformInterceptor	Interceptor	Enveloppe chaque reponse dans un format uniforme { data, meta }
HttpExceptionFilter	Filter	Centralise la gestion des erreurs et formate les reponses d'erreur
ThrottlerGuard	Guard	Rate limiting : 100 requetes / minute par IP (configurable)
CorrelationIdMiddleware	Middleware	Injecte un trace_id unique dans chaque requete pour le suivi distribue

4. Application Mobile React Native — Architecture Offline-First

4.1 Architecture generale

L'application mobile suit une architecture en couches basee sur Zustand (gestion d'etat) + React Query (cache serveur) + WatermelonDB (persistance locale), adaptee aux applications React Native de production. Cette architecture separe clairement l'interface utilisateur de la logique metier, facilitant les tests unitaires et la maintenabilite.

Couche	Librairies	Responsabilite
Presentation (UI)	React Native Core Components, React Navigation 6	Ecrans, composants visuels, animations, navigation
Store / Hooks	Zustand + React Query (TanStack)	Gestion d'etat, logique de presentation, gestion des evenements
Repository	Custom classes	Abstraction des sources de donnees, logique de synchronisation
Data Sources	WatermelonDB (SQLite), Axios, react-native-audio-recorder-player	BDD locale, appels API, capture audio
Dependency Injection	React Native MMKV + Dependency Injection via Context	Injection de dependances, singletons, services

4.2 Gestion du mode offline

L'architecture offline-first est le coeur technique de l'application mobile. Elle garantit un fonctionnement continu meme sans connexion reseau, particulierement important dans des zones a couverture reseau instable.

Mecanisme de capture offline

- La capture audio est demarree en background service (react-native-background-fetch (iOS + Android))
- Chaque extrait capture est encode (WAV 44100Hz, mono, 45 sec max) et stocke dans SQLite local (WatermelonDB)
- Un worker tourne en arriere-plan toutes les 30 secondes pour verifier la connectivite reseau
- Des la detection d'une connexion, la file d'attente locale est synchronisee par lot de 50 captures max vers POST /audio/sync
- Chaque capture synchronisee est marquee 'synced' en base locale pour eviter les doublons
- La base locale garde un historique de 7 jours maximum (auto-purge) pour gerer l'espace disque
- En cas d'echec de synchronisation, un mecanisme de retry exponentiel est applique (1s, 2s, 4s, 8s...)

4.3 Packages React Native utilises

Package	Version	Usage dans HMIS
react-native-audio-recorder-player	^3.5.3	Capture microphone, encodage AAC/WAV, gestion du buffer audio
@tanstack/react-query	^5.17.0	Cache serveur, synchronisation API, invalidation automatique
zustand	^4.5.0	Gestion etat globale (auth, config, preferences), stores reactifs

Package	Version	Usage dans HMIS
@nozbe/watermelondb	^0.27.1	ORM SQLite local haute performance, persistance offline
axios	^1.6.7	Client HTTP, interceptors JWT auto-refresh, upload multipart
react-native-background-fetch	^4.2.3	Taches de fond iOS et Android, sync periodique offline
react-native-keychain	^8.2.0	Stockage securise des tokens JWT (iOS Keychain / Android Keystore)
react-native-permissions	^3.10.1	Gestion permissions runtime (microphone, localisation, notifs)
@react-native-community/netinfo	^11.3.1	Detection connectivite reseau en temps reel
react-native-geolocation-service	^5.3.1	Localisation GPS pour enregistrement position etablisement
react-navigation	^6.1.10	Navigation entre ecrans, deep linking, Stack et Tab navigators
react-native-mmkv	^2.11.0	Stockage cle-valeur ultra-rapide, cache leger, preferences
react-native-localize	^3.0.6	Internationalisation, detection locale, formatage dates

5. Frontend Web — Dashboard et Reporting

5.1 Architecture Next.js

Le dashboard web est construit avec Next.js 14 en mode App Router. Il utilise le Server-Side Rendering (SSR) pour les pages publiques et le Client-Side Rendering (CSR) pour les parties interactives du dashboard, optimisant ainsi les performances et le SEO.

Concept	Technologie	Application dans HMIS
Routing	Next.js App Router	Navigation entre dashboard, etablissements, rapports, administration
State Management	Zustand + React Query	Zustand pour l'etat global (auth, preferences), React Query pour le cache serveur
Composants UI	Tailwind CSS + shadcn/ui	Design system coherent, composants accessibles, theme HMIS personnalise
Visualisations	Recharts + Leaflet.js	Graphiques d'evolution, camemberts, carte geographique interactive
Formulaires	React Hook Form + Zod	Validation cote client, gestion des erreurs, UX optimisee
Export PDF	jsPDF + html2canvas	Generation client du rapport PDF (complementaire au PDF serveur)
Export Excel	SheetJS (xlsx)	Export des donnees brutes en .xlsx directement depuis le navigateur
HTTP Client	Axios + React Query	Requetes API, cache, invalidation automatique, optimistic updates
Authentification	NextAuth.js	Gestion des sessions, refresh automatique du token JWT

5.2 Pages principales du dashboard

Page / Route	Composants cles	Donnees consommees
/dashboard	KPICards, DiffusionsChart, TopMusiquesList, AlertesWidget	GET /dashboard/kpis, /dashboard/evolution
/dashboard/carte	MapComponent (Leaflet), EtablissementPopup, FiltersPanel	GET /dashboard/carte
/dashboard/musiques	TopMusiquesTable, GenresPieChart, ArtistesBarChart	GET /dashboard/top-musiques
/etablissements	EtablissementsTable, StatusBadge, SearchBar, Pagination	GET /etablissements
/etablissements/[id]	ProfilCard, DiffusionsHistory, StatsPanel, StatutActions	GET /etablissements/:id + /diffusions
/rapports	RapportForm, JobsList, DownloadButton, ProgressBar	POST + GET /rapports
/administration	UsersTable, RolesManager, SystemConfig	GET /utilisateurs (admin only)

6. Base de Donnees PostgreSQL — Modele de Donnees

6.1 Entites principales

La base de donnees PostgreSQL centralise toutes les donnees du systeme. Le modele est normalise (3NF) pour garantir l'integrite des donnees tout en restant performant pour les requetes analytiques grace a des index strategiques.

Table : utilisateurs

Colonne	Type	Contraintes	Description
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Identifiant unique
nom	VARCHAR(255)	NOT NULL	Nom complet
email	VARCHAR(255)	UNIQUE, NOT NULL	Email de connexion
telephone	VARCHAR(20)	UNIQUE	Numero de telephone (OTP)
mot_de_passe_hash	VARCHAR(255)	NOT NULL	Hash bcrypt du mot de passe
role	ENUM	NOT NULL : admin etablissement partenaire	Role RBAC
actif	BOOLEAN	NOT NULL, DEFAULT true	Compte actif/inactif
telephone_verifie	BOOLEAN	DEFAULT false	OTP valide
refresh_token_hash	VARCHAR(255)	NULLABLE	Hash du refresh token actif
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	Date de creation
updated_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	Derniere mise a jour

Table : etablissements

Colonne	Type	Contraintes	Description
id	UUID	PRIMARY KEY	Identifiant unique
utilisateur_id	UUID	FK -> utilisateurs.id, NOT NULL	Proprietaire du compte
nom	VARCHAR(255)	NOT NULL	Nom de l'etablissement
type	ENUM	NOT NULL : bar maquis cave boite_nuit ...	Type d'etablissement
adresse	TEXT	NOT NULL	Adresse complete
commune	VARCHAR(100)	NOT NULL	Commune
ville	VARCHAR(100)	NOT NULL, DEFAULT 'Abidjan'	Ville
latitude	DECIMAL(10,7)	NULLABLE	Coordonnee GPS latitude
longitude	DECIMAL(10,7)	NULLABLE	Coordonnee GPS longitude

Colonne	Type	Contraintes	Description
telephone	VARCHAR(20)	NULLABLE	Contact direct
capacite_accueil	INTEGER	NULLABLE, CHECK > 0	Capacite max en personnes
statut	ENUM	NOT NULL, DEFAULT 'en_attente'	en_attente actif suspendu inactif
valide_par	UUID	FK -> utilisateurs.id, NULLABLE	Admin ayant valide
date_validation	TIMESTAMPTZ	NULLABLE	Date de validation
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	Date de creation

Table : diffusions

Colonne	Type	Contraintes	Description
id	UUID	PRIMARY KEY	Identifiant unique
etablissement_id	UUID	FK -> etablissements.id, NOT NULL	Etablissement source
capture_id	UUID	FK -> captures.id, NOT NULL	Capture audio source
timestamp_diffusion	TIMESTAMPTZ	NOT NULL	Horodatage exact de la diffusion
duree_secondes	INTEGER	NOT NULL, CHECK > 0	Duree de l'extrait identifie
score_confiance	DECIMAL(4,3)	NOT NULL, CHECK 0-1	Score de fiabilite de la reconnaissance
titre	VARCHAR(500)	NOT NULL	Titre de la musique
artiste	VARCHAR(500)	NOT NULL	Artiste / groupe
album	VARCHAR(500)	NULLABLE	Album
isrc	VARCHAR(20)	NULLABLE	Code ISRC standard
label	VARCHAR(255)	NULLABLE	Label musical
genre	VARCHAR(100)	NULLABLE	Genre musical
annee_sortie	SMALLINT	NULLABLE	Annee de sortie
source_api	ENUM	NOT NULL : acrccloud audd	API source de la reconnaissance
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	Date d'insertion

6.2 Index strategiques

Les index suivants sont essentiels pour les performances des requetes analytiques du dashboard et du reporting :

Index	Table	Colonnes	Justification
idx_diffusions_etablissement_ts	diffusions	etablissement_id, timestamp_diffusion DESC	Requetes historique par etablissement (tres frequentes)

Index	Table	Colonnes	Justification
idx_diffusions_artiste	diffusions	artiste, timestamp_diffusion DESC	Top artistes, rapports par artiste
idx_diffusions_isrc	diffusions	isrc, timestamp_diffusion	Suivi des oeuvres par code ISRC
idx_etablissements_statut	etablissements	statut, ville, commune	Filtres dashboard, carte geo
idx_etablissements_geo	etablissements	latitude, longitude (spatial)	Requetes geospatiales (PostGIS futur)
idx_captures_etablissement	captures	etablissement_id, statut, created_at	Suivi des captures en attente
idx_utilisateurs_email	utilisateurs	email (unique)	Lookup login (critique)

7. Infrastructure Cloud — AWS / Google Cloud

7.1 Schema d'infrastructure

L'infrastructure est entièrement cloud-managée. Voici la décomposition des services cloud utilisés et leur rôle dans l'architecture HMIS :

Service	AWS	GCP equivalent	Configuration HMIS
Serveur application	EC2 t3.medium (2vCPU, 4GB RAM)	e2-medium	Auto Scaling Group min:1 max:3, Ubuntu 22.04 LTS
Base de données	RDS PostgreSQL 15 (db.t3.medium)	Cloud SQL PostgreSQL 15	Multi-AZ, sauvegardes auto 7j, chiffrement at-rest
Stockage objets	S3 (Standard + Intelligent-Tiering)	Cloud Storage	Versioning actif, lifecycle rules, CORS configure
CDN	CloudFront	Cloud CDN	Distribution assets frontend, cache statique 24h
Load Balancer	ALB (Application Load Balancer)	Cloud Load Balancing	HTTPS termination, health checks /health
DNS	Route 53	Cloud DNS	Zones : hmis-project.ci, api.hmis-project.ci
Secrets	AWS Secrets Manager	Secret Manager	DB credentials, JWT secrets, API keys ACRCLOUD
Monitoring	CloudWatch	Cloud Monitoring	Alerte CPU > 80%, mémoire > 85%, latence > 1s
Email	SES (Simple Email Service)	SendGrid	Emails transactionnels, reset mot de passe
File de messages	SQS (ou Bull+Redis)	Cloud Pub/Sub	File de traitement des captures audio, rapports async

7.2 Configuration Docker et Docker Compose

Tous les services sont containerisés via Docker pour garantir des environnements reproductibles. Le fichier docker-compose.yml orchestre les services en développement local :

```
# docker-compose.yml
services:
  api:
    build: ./backend
    ports: ['3000:3000']
    environment:
      - DATABASE_URL=postgresql://postgres:secret@db:5432/hmis
      - JWT_SECRET=${JWT_SECRET}
      - ACRCLOUD_HOST=${ACRCLOUD_HOST}
    depends_on: [db, redis]

  db:
    image: postgres:15-alpine
    environment: {POSTGRES_DB: hmis, POSTGRES_PASSWORD: secret}
    volumes: ['pgdata:/var/lib/postgresql/data']
```

```
redis:  
  image: redis:7-alpine  
  ports: ['6379:6379']  
  
swagger:  
  image: swaggerapi/swagger-ui  
  environment: {SWAGGER_JSON: /app/hmis-api.yaml}  
  ports: ['8080:8080']
```

8. Securite et Authentification

8.1 Flux d'authentification complet

Le systeme HMIS implemente une authentification double facteur (2FA) pour les etablissements et un JWT a courte duree de vie avec refresh token. Le schema suivant illustre le flux complet :

Etape	Action	Composant	Detail technique
1	Inscription	POST /auth/register	Hash bcrypt du mot de passe (cost 12), creation user en base
2	Envoi OTP	POST /auth/otp/envoyer	Code 6 chiffres, expire 10 min, stocke en Redis (hash SHA256)
3	Verification OTP	POST /auth/otp/verifier	Comparaison hash, activation compte, premier JWT emis
4	Connexion	POST /auth/login	Verification bcrypt, emission access_token (15 min) + refresh_token (7j)
5	Requete protegee	Tout endpoint avec BearerAuth	JwtAuthGuard valide signature + expiration + role
6	Renouvellement	POST /auth/refresh	Validation refresh_token hash en base, nouveau access_token emis
7	Deconnexion	POST /auth/logout	Invalidation refresh_token en base (hash efface)

8.2 Mesures de securite implementees

Categorie	Mesure	Implementation
Transport	HTTPS obligatoire (TLS 1.3)	Certificat Let's Encrypt auto-renouvele, HSTS active, redirect HTTP->HTTPS
Donnees au repos	Chiffrement AES-256	RDS chiffrement active, S3 SSE-S3, Secrets Manager pour les cles
Authentification	JWT HS256 + bcrypt	Access token 15min, refresh 7j, bcrypt cost 12, rotation des secrets
Autorisation	RBAC strict	3 roles : admin, etablissement, partenaire — guards NestJS sur chaque route
Validation	class-validator + Zod	Sanitization de toutes les entrees, rejet des payloads non conformes
Rate limiting	Throttler NestJS	100 req/min global, 5 req/min sur /auth/login, 3 req/min sur /auth/otp
OWASP Top 10	Headers securite (Helmet.js)	CSP, X-Frame-Options, HSTS, X-XSS-Protection via @nestjs/helmet
Injection SQL	TypeORM parametrisé	Zero SQL brut, requetes toujours via ORM avec parametres lies
Logs securite	Audit trail complet	Chaque action sensible logee avec user_id, IP, timestamp, action
Secrets	Variables d'environnement	Aucun secret en dur dans le code, rotation possible via Secrets Manager

9. Integration API de Reconnaissance Musicale

9.1 Architecture d'integration

L'integration de l'API de reconnaissance musicale est encapsulee dans un module dedie (ReconnaissanceModule) via le design pattern Adapter, permettant de changer de fournisseur (ACRCloud -> AudD) sans modifier le reste du code.

Pattern Adapter — ReconnaissanceService

- Interface `IMusicRecognitionAdapter` definit le contrat : `recognize(audioBuffer: Buffer): Promise<RecognitionResult>`
- `AcrCloudAdapter` implemente cette interface pour ACRCloud (signature HMAC-SHA1, multipart/form-data)
- `AudDAdapter` implemente la meme interface pour AudD (token API, JSON payload)
- `ReconnaissanceService` injecte l'adapter configure via la variable d'env `RECOGNITION_PROVIDER`
- En cas de non-reconnaissance (score < seuil), un second appel peut etre tente sur l'autre provider
- Tous les resultats sont normalises dans un format interne unifie avant stockage en base

9.2 Flux technique de reconnaissance

Etape	Service	Detail
1. Reception	AudioController	Validation du fichier audio (format, taille, duree via ffprobe)
2. Stockage temp	AudioService	Upload S3 avec cle temporaire (TTL 24h), retour capture_id
3. Queue	Bull Queue (Redis)	Job ajoute a la queue 'recognition', traitement asynchrone
4. Worker	ReconnaissanceWorker	Recupere le fichier S3, prepare le buffer audio
5. Appel ACRCloud	AcrCloudAdapter	POST multipart avec signature HMAC, timeout 10s, retry 2x
6. Parse reponse	ReconnaissanceService	Extraction metadata (titre, artiste, ISRC, score), normalisation
7. Stockage log	DiffusionsService	INSERT en base PostgreSQL si score >= seuil minimum (0.60)
8. Cleanup	AudioService	Suppression du fichier temporaire S3 apres 24h (lifecycle rule)
9. Notification	WebSocket (optionnel)	Notification temps reel au dashboard via Socket.io

9.3 Gestion des erreurs de reconnaissance

Scenario	Comportement	Log
Score < seuil (0.60)	Diffusion NON enregistree, capture marquee 'non_identifie'	WARNING avec score et etablissement_id
Timeout API (>10s)	Retry 2x avec backoff exponentiel, puis echec	ERROR avec duree, provider, capture_id
API indisponible	Bascule automatique sur provider de secours (AudD)	WARN + alert Sentry
Musique inconnue	Capture marquee 'non_identifie', metadata null	INFO — pas d'erreur
Format audio invalide	Rejet immediat, erreur 400 retournee au client	ERROR avec format detecte
Quota API depasse	File d'attente mise en pause, alerte admin	CRITICAL + PagerDuty

10. Pipeline CI/CD et DevOps

10.1 Pipeline GitHub Actions

Le pipeline CI/CD est entièrement automatisé via GitHub Actions. Chaque push sur une branche déclenche la chaîne de validation. Seule la branche main est déployée en production.

Stage	Declencheur	Etapes	Duree estimee
Lint & Format	Tout push	ESLint, Prettier check, TypeScript strict compile	~2 min
Tests unitaires	Tout push	Jest, couverture > 70%, rapport HTML genere	~5 min
Tests integration	PR vers main	Supertest sur BDD PostgreSQL de test (Docker)	~8 min
Build Docker	PR mergee vers main	docker build, tag avec commit SHA + 'latest'	~4 min
Push Registry	Apres build OK	Push image vers AWS ECR / GCR, scan vulnerabilites Trivy	~3 min
Deploy Staging	Apres push registry	Deploy automatique sur EC2 staging, smoke tests	~5 min
Tests E2E	Apres deploy staging	Playwright sur staging — parcours critiques valides	~10 min
Deploy Production	Manuel (bouton)	Deploy rolling sur EC2 prod, zero-downtime, rollback auto si echec	~5 min

10.2 Strategie de branches

Branche	Acces	Regles
main	Tech Lead uniquement	Protection stricte, PR obligatoire, 2 reviewers, CI vert requis
develop	Toute l'equipe	Integration continue, merge depuis les feature branches
feature/xxx	Developpeur assigne	Branche depuis develop, naming : feature/nom-fonctionnalite
hotfix/xxx	Tech Lead	Correction urgente production, merge vers main ET develop
release/x.x.x	Tech Lead	Preparation release, freeze des features, derniers tests

11. Monitoring, Logs et Alertes

11.1 Stack de monitoring

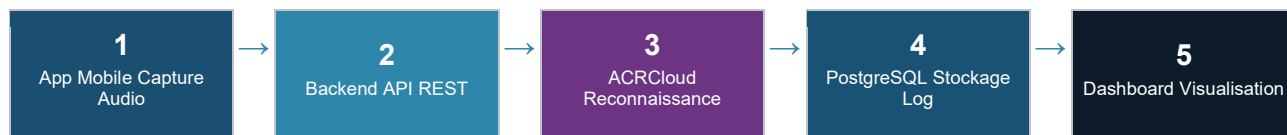
Outil	Role	Metriques surveillees
Sentry	Error tracking applicatif	Exceptions backend/frontend, performance, release tracking
Grafana + Prometheus	Metriques infrastructure et applicatives	CPU, RAM, latence API, throughput, nb diffusions/min
CloudWatch / GCP Monitoring	Monitoring infrastructure cloud	EC2, RDS, S3, ALB, couts cloud
Winston (backend)	Logs applicatifs structures	Requetes HTTP, erreurs, actions metier, evenements securite
Datadog APM (optionnel)	Distributed tracing	Traces distribuees entre les services, bottlenecks

11.2 Alertes critiques configurees

Alerte	Seuil	Canal	Priorite
Latence API p95 > 1s	3 mesures consecutives	Slack #alertes-tech	WARNING
Taux d'erreur HTTP 5xx > 5%	Sur 5 minutes glissantes	Slack + Email tech lead	CRITICAL
CPU EC2 > 85%	5 minutes consecutives	CloudWatch + Slack	WARNING
Espace disque BDD > 80%	Detection immediate	Email + PagerDuty	CRITICAL
Quota API ACRCLOUD > 90%	Par heure	Email admin HANKES	WARNING
Echec CI/CD	Apres 2 retries	Slack #deploys	WARNING
Taux reconnaissance < 70%	Sur 1 heure glissante	Email admin HANKES	WARNING
Aucune diffusion depuis 2h (etablissement actif)	Detection continue	Dashboard admin	INFO

12. Flux de Donnees Complets

12.1 Flux principal — Capture et reconnaissance en temps reel



Etape	Composant	Protocol	Donnees echangees
1. Capture audio	App Mobile (React Native)	Systeme (micro)	Buffer audio WAV, 44100Hz, mono, 45s max
2. Upload extrait	App -> Backend	HTTPS POST multipart	File audio + etablissement_id + timestamp + duree
3. Mise en queue	Backend -> Bull/SQS	Redis / AWS SQS	Job avec capture_id, chemin S3, metadata
4. Appel reconnaissance	Worker -> ACRCLOUD	HTTPS POST (HMAC)	Buffer audio signe, reponse JSON (titre, artiste, ISRC, score)
5. Stockage diffusion	Worker -> PostgreSQL	TypeORM INSERT	Log complet : timestamp, musique, score, etablissement_id
6. Consultation dashboard	Admin -> Backend	HTTPS GET REST	Requetes analytiques, agregations, pagination

12.2 Flux offline — Synchronisation differee

Etape	Composant	Stockage
Capture sans connexion	App Mobile capture et encode l'extrait	SQLite local (WatermelonDB) — jusqu'a 200 captures
Detection reconnexion	NetInfo detecte le retour reseau	Trigger react-native-background-fetch
Preparation batch	SyncService (TypeScript) regroupe jusqu'a 50 captures	Queue locale priorisee par timestamp_capture
Envoi batch	POST /audio/sync avec captures[] base64	Backend valide, deduplique (reference_locale), met en queue
Traitement asynchrone	Bull Worker traite chaque capture individuellement	Meme flux que temps reel (etapes 4 et 5)
Confirmation sync	Reponse 207 avec statut par capture	App marque les captures 'synced' en SQLite local

13. Decisions d'Architecture — ADR

Les Architecture Decision Records (ADR) documentent les choix techniques majeurs, leurs justifications et les alternatives considerees.

ADR-001 — Choix de React Native comme framework mobile *[Accepte]*

Contexte	Besoin d'une application mobile cross-platform (Android + iOS) partageant une meme codebase, avec acces aux APIs natives (microphone, background, stockage securise).
Decision	React Native 0.73+ avec TypeScript strict.
Justification	L'equipe de developpement possede une expertise JavaScript/TypeScript deja utilisee pour le backend (NestJS). React Native permet de partager 90% du code entre Android et iOS. Expo (bare workflow) facilite la configuration native tout en permettant des modules natifs personnalisés. Alternative React Native non retenue car elle aurait necessite l'apprentissage de TypeScript, une nouvelle technologie sans synergie avec le reste du stack.
Consequences	Build natif necessaire pour les modules natifs (microphone background). Performances legerement inferieures a React Native sur animations complexes, sans impact pour l'usage HMIS (capture audio passive).

ADR-001b — Choix de NestJS comme framework backend *[Accepte]*

Contexte	Besoin d'un framework Node.js structure, typise et maintenable pour une equipe de 2 developpeurs backend.
Decision	NestJS avec TypeScript strict.
Justification	Architecture modulaire native, injection de dependances, decorateurs, support TypeORM, Swagger integre, vaste ecosysteme. Alternative Express trop bas niveau pour ce contexte.
Consequences	Courbe d'apprentissage de 1-2 jours pour les devs non familiers avec NestJS, gain majeur en maintenabilite long terme.

ADR-002 — Architecture offline-first pour l'application mobile *[Accepte]*

Contexte	Les etablissements sont situes dans des zones a couverture reseau variable (Cote d'Ivoire). Une connexion instable ne doit pas interrompre la capture.
Decision	SQLite local (WatermelonDB) + synchronisation batch differee.
Justification	Garantit 100% de fiabilite de capture independamment du reseau. Synchronisation automatique transparente via react-native-background-fetch.
Consequences	Complexite supplementaire (gestion conflits, deduplication), mais experience utilisateur nettement superieure.

ADR-003 — Utilisation d'ACRCloud comme fournisseur principal *[Accepte]*

Contexte	Besoin d'identifier des musiques africaines (ivoiriennes, camerounaises, senegalaises...) avec une precision > 80%.
Decision	ACRCloud comme fournisseur principal, AudD comme fallback.
Justification	ACRCloud offre la meilleure couverture musicale africaine de sa categorie, une API documentee et une latence < 2s. Leur base inclut > 100M pistes.

Consequences	Cout variable selon volume de requetes. Dependance a un service tiers — mitigue par le pattern Adapter.
---------------------	---

ADR-004 — PostgreSQL comme unique base de donnees *[Accepte]*

Contexte	Evaluation entre PostgreSQL, MongoDB et une architecture polyglotte.
Decision	PostgreSQL uniquement, avec TypeORM.
Justification	Les donnees sont structurees et relationnelles. PostgreSQL excelle en requetes analytiques (fenetres, agregations). ACID garantit l'integrite des logs de diffusion. Pas de besoin NoSQL identifie.
Consequences	Schema migratoire obligatoire a chaque evolution. Avantage : une seule technologie a maitriser et maintenir.

14. Glossaire Technique

Terme	Définition
ACRCloud	Audio Content Recognition Cloud — API de reconnaissance musicale utilisée comme fournisseur principal dans HMIS
ADR	Architecture Decision Record — document qui trace les décisions architecturales importantes et leurs justifications
ALB	Application Load Balancer (AWS) — répartiteur de charge au niveau applicatif (Layer 7)
AudD	API de reconnaissance musicale alternative à ACRCloud, utilisée comme fallback dans HMIS
Zustand/Flux	Business Logic Component — pattern d'architecture React Native séparant UI et logique métier
Bull Queue	Librairie Node.js de gestion de files d'attente basée sur Redis, utilisée pour le traitement asynchrone des captures audio
CDN	Content Delivery Network — réseau de distribution de contenu permettant de servir les assets web depuis le serveur le plus proche de l'utilisateur
CI/CD	Continuous Integration / Continuous Deployment — pratique d'automatisation du build, test et déploiement
Drift	ORM React Native pour SQLite, utilisé pour la persistance locale offline dans l'app mobile HMIS
EC2	Elastic Compute Cloud (AWS) — service de serveurs virtuels utilisé pour héberger le backend NestJS
HMAC-SHA1	Hash-based Message Authentication Code — algorithme de signature utilisé par ACRCloud pour authentifier les requêtes
ISRC	International Standard Recording Code — code unique identifiant un enregistrement musical, utilisé pour la traçabilité des œuvres
JWT	JSON Web Token — standard ouvert pour la transmission sécurisée d'informations sous forme de token signé
NestJS	Framework Node.js progressif, basé sur TypeScript et l'architecture modulaire, utilisé comme backend HMIS
ORM	Object-Relational Mapping — technique de programmation permettant de manipuler une BDD relationnelle via des objets (TypeORM dans HMIS)
OTP	One-Time Password — code à usage unique envoyé par SMS pour la vérification du numéro de téléphone
RBAC	Role-Based Access Control — modèle de contrôle d'accès basé sur les rôles (admin, établissement, partenaire dans HMIS)
RDS	Relational Database Service (AWS) — service de base de données managée utilisé pour PostgreSQL dans HMIS
S3	Simple Storage Service (AWS) — stockage objet utilisé pour les fichiers audio temporaires, rapports et sauvegardes
SLA	Service Level Agreement — niveau de service garanti (ex : 99.9% de disponibilité)
SSR	Server-Side Rendering — génération HTML côté serveur pour le dashboard Next.js

Terme	Definition
TLS 1.3	Transport Layer Security 1.3 — protocole de chiffrement des communications reseau, successeur de SSL
TypeORM	ORM TypeScript/Node.js utilise dans le backend NestJS pour interagir avec PostgreSQL
WAV	Waveform Audio File Format — format audio non compresse utilise pour les extraits captures (meilleure qualite de reconnaissance)

Document d'Architecture Technique HMIS — v1.0 — Mars 2026*Confidentiel — Usage exclusivement reserve a HANKES*contact@hmis-project.ci